Computación y estructuras discretas I

Unidad 3: Estructuras Discretas no recursivas y análisis de algoritmos – Colas de prioridad

Juan Marcos Caicedo Mejía – jmcaicedo@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes
Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas
Universidad ICESI
Material adaptado del material original del profesor Oscar Bedoya

2026

Tabla de contenido

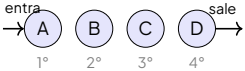
1. Colas de prioridad

De Cola FIFO a Cola de prioridad

¿Qué cambia y qué se conserva?

Cola FIFO

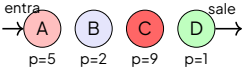
El orden de salida es **el orden de llegada**.



A saldrá siempre antes que D,
sin importar nada más.

Cola de prioridad

El orden de salida es **la prioridad asignada**.



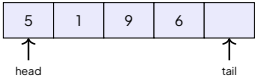
C sale primero (p=9),
aunque llegó en 3er lugar.

Lo que cambia

`DEQUEUE` (saca el frente) → `EXTRACT-MAX` (saca el de mayor prioridad)
no necesita comparar → compara prioridades
orden implícito (posición) → orden explícito (campo `prioridad`)

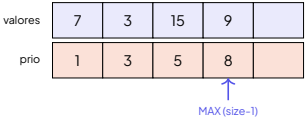
Estructura interna: Cola vs Cola de prioridad

Cola — arreglo circular



- Un solo arreglo de `int`
- El orden físico **es** el orden lógico
- `head/tail` marcan inicio y fin
- Extrae siempre por `head`

Cola de prioridad — dos arreglos paralelos



- **Dos** arreglos paralelos sincronizados
- Ordenados ascendente por prioridad
- El máximo siempre en `size-1`
- Extrae siempre por `size-1`

Clave

En la Cola normal el orden viene de la **posición** (`head/tail`). En la Cola de prioridad el orden viene de un **campo de prioridad** explícito y el arreglo se mantiene siempre **ordenado**.

INSERT paso a paso — estado inicial

4 elementos insertados, ordenados por prioridad ascendente

	[0]	[1]	[2]	[3]	[4]	
`valores`	20	8	15	3	–	`size=4`
`prio`	1	2	5	7	–	

ordenado ascendente por prioridad

`size-1` → MAX (p=7)

Queremos insertar **valor=12, prioridad=4**.

1. Recorremos de derecha a izquierda buscando dónde insertar.
2. $p = 4$ es mayor que $p[1] = 2$ pero menor que $p[2] = 5$.
3. Todo elemento con $p \geq 4$ se **desplaza un lugar a la derecha**.
4. El hueco resultante en el índice 2 recibe el nuevo par.

INSERT paso a paso — desplazamiento e insei

Bucle: `while (i >= 0 && prioridades[i] > p)`

Pseudocódigo del desplazamiento

```
i = size - 1
while (i >= 0 && prioridades[i] > p) {
  valores[i+1] = valores[i]
  prioridades[i+1] = prioridades[i]
  i--
}
valores[i+1] = nuevoValor
prioridades[i+1] = p
size++
```

Contenido

1. Colas de prioridad

Colas de prioridad

Motivación

En una cola FIFO el orden de atención depende **únicamente** del momento de llegada. Sin embargo, en muchos sistemas reales algunos elementos deben atenderse antes que otros independientemente de cuándo llegaron.

Ejemplos:

- Sistema operativo: procesos con mayor prioridad obtienen la CPU primero
- Sala de urgencias: pacientes críticos se atienden antes que leves
- Algoritmos de grafos: Dijkstra, Prim extraen siempre el nodo de menor costo

Cola de prioridad

Estructura de datos en la que cada elemento tiene asociada una **prioridad**, y la operación de extracción siempre devuelve el elemento de **mayor prioridad**, sin importar el orden de inserción.

Colas de prioridad

TAD Cola de prioridad — operaciones

`PRIORITY-QUEUE-EMPTY(PQ)`	retorna verdadero si la cola no tiene elementos
`INSERT(PQ, x, p)`	inserta el valor *x* con prioridad *p*
`MAXIMUM(PQ)`	retorna el valor de mayor prioridad *sin extraerlo*
`EXTRACT-MAX(PQ)`	extrae y retorna el valor de mayor prioridad

Invariante central

La cola garantiza que `EXTRACT-MAX` siempre entrega el elemento con la prioridad más alta, independientemente del orden de inserción.

Colas de prioridad

Implementación con arreglo ordenado

Se usan dos arreglos paralelos de tamaño n, mantenidos en orden **ascendente** de prioridad: el elemento de **mayor** prioridad ocupa siempre el índice `size` − 1.

	0	1	2	3	4
`valores`	15	8	3	12	20
`prioridades`	1	2	4	5	9

`size-1` ← MAXIMUM

- El arreglo siempre está ordenado: $p[0] \leq p[1] \leq \cdots \leq p[\texttt{size} - 1]$
- `EXTRACT-MAX`: toma el último elemento — $O(1)$
- `INSERT`: desplaza hacia la derecha hasta encontrar la posición — $O(n)$

Colas de prioridad

```
INSERT(PQ, x, p)
if size[PQ] == n
then error ``overflow''
else i = size[PQ] − 1
     while i ≥ 0 and prioridad[PQ][i] > p
           valores[PQ][i+1] = valores[PQ][i]
           prioridad[PQ][i+1] = prioridad[PQ][i]
           i = i − 1
     valores[PQ][i+1] = x
     prioridad[PQ][i+1] = p
     size[PQ] = size[PQ] + 1
```

Colas de prioridad

```
INSERT(PQ, x, p)
if size[PQ] == n
then error ``overflow''
else i = size[PQ] − 1
     while i ≥ 0 and prioridad[PQ][i] > p
           valores[PQ][i+1] = valores[PQ][i]
           prioridad[PQ][i+1] = prioridad[PQ][i]
           i = i − 1
     valores[PQ][i+1] = x
     prioridad[PQ][i+1] = p
     size[PQ] = size[PQ] + 1
```

$T(n) = O(n)$ — el bucle desplaza hasta n elementos en el peor caso

Colas de prioridad

```
MAXIMUM(PQ)
if size[PQ] == 0
then error ``underflow''
else return
        valores[PQ][size[PQ]]
```

```
EXTRACT-MAX(PQ)
if size[PQ] == 0
then error ``underflow''
else x = valores[PQ][size[PQ]]
     size[PQ] = size[PQ] — 1
     return x
```

Colas de prioridad

```
MAXIMUM(PQ)
if size[PQ] == 0
then error ``underflow''
else return
        valores[PQ][size[PQ]]
```

$T(n) = O(1)$

```
EXTRACT-MAX(PQ)
if size[PQ] == 0
then error ``underflow''
else x = valores[PQ][size[PQ]]
        size[PQ] = size[PQ] − 1
        return x
```

$T(n) = O(1)$

Clave de la implementación

Como el arreglo está ordenado ascendente, el máximo siempre está en la última posición (`size-1`). Extraerlo solo requiere decrementar `size` — sin desplazamientos.

Implementación en Java

ColaPrioridad.java — estructura

```java
public class ColaPrioridad {
  private int[] valores;
  private int[] prioridades;
  private int size;
  private int size_max;

  public ColaPrioridad(int n) {
    this.size_max    = n;
    this.size        = 0;
    this.valores     = new int[n];
    this.prioridades = new int[n];
  }

  public boolean isEmpty() {
    return size == 0;
  }
}
```

- Dos arreglos paralelos de tamaño n: uno para valores, otro para prioridades
- Ambos se mantienen sincronizados: `valores[i]` tiene prioridad `prioridades[i]`
- Orden ascendente por prioridad: índice `size-1` $\rightarrow$ mayor prioridad

Implementación en Java

ColaPrioridad.java — insert

```java
public void insert(int x, int p) {
  if (size == size_max)
    throw new RuntimeException("Overflow");

  int i = size - 1;
  while (i >= 0 && prioridades[i] > p) {
    valores[i + 1]      = valores[i];
    prioridades[i + 1] = prioridades[i];
    i--;
  }
  valores[i + 1]      = x;
  prioridades[i + 1] = p;
  size++;
}
```

- Recorre hacia la izquierda desplazando elementos de mayor prioridad
- Inserta x en la posición correcta para mantener el orden
- Sincroniza **ambos** arreglos en cada desplazamiento
- Costo: $O(n)$

Implementación en Java

ColaPrioridad.java — extractMax y maximum

```java
public int maximum() {
  if (size == 0)
    throw new NoSuchElementException();
  return valores[size - 1];
}

public int extractMax() {
  if (size == 0)
    throw new NoSuchElementException();
  int val = valores[size - 1];
  size--;
  return val;
}
```

- `maximum`: solo lee `valores[size-1]` — no modifica nada
- `extractMax`: lee el último elemento y decrementa `size`; **no hay desplazamiento**
- Ambos: $O(1)$

Operación	Costo
`isEmpty`	$O(1)$
`maximum`	$O(1)$
`extractMax`	$O(1)$
`insert`	$O(n)$